Guía de ejecución de Compiscript MIPS

Índice

Elegí por dónde empezar . . . 1
Requisitos y preparación . . . 1
Ejecutar un ejemplo completo . . . 1
CLI y reportes . . . 2
IDE . . . 2
Arquitectura y alcance . . . 2
Diagnóstico de problemas frecuentes . . . 3
Límites explícitos . . . 3
Pruebas y presentación . . . 4

Compiscript ahora puede traducir `.cps` a ensamblador MIPS para MARS. El frontend ANTLR, la semántica y el TAC existentes siguen siendo la entrada del backend; compilar no ejecuta el programa. Esta fase es independiente del intérprete SQL que el repositorio también llama «Entrega 3».

Elegí por dónde empezar

Si necesitás…	Documento
Entender qué se entregó y cómo se relaciona con la rúbrica	Informe técnico
Explicar memoria, pila, registros, llamadas y runtime	Arquitectura
Seguir ejemplos desde el TAC hasta MIPS	Referencia TAC a MIPS
Ver resultados realmente observados y límites de la evidencia	Aceptación
Consultar los requisitos originales	Enunciado transcrito
Leer o entregar las cuatro copias PDF	Paquete de lectura

Requisitos y preparación

Desde la raíz, comprobá `go version` y `java -version`. `go.mod` define el mínimo Go; las dependencias del módulo deben estar disponibles. Java ejecuta MARS: no se instala automáticamente con el script. No hace falta regenerar ANTLR para usar la CLI/IDE con las fuentes generadas ya presentes.

El instalador de MARS usa `bash`, `curl` y `shasum`, descarga el JAR oficial solo si falta y verifica el hash fijado. Se guarda dentro de `tools/mars/`, sin instalación global ni cambios a dependencias Go. La primera descarga requiere red; la ejecución posterior no. La guía de la herramienta explica procedencia, checksum y sus límites de confianza.

Ejecutar un ejemplo completo

Requisitos: Go según `go.mod`, Java en PATH y las herramientas de shell descritas en la guía de MARS.

Desde la raíz:

```
bash scripts/install-mars.sh
bash scripts/test-mars.sh
bash scripts/test-mips.sh
```

La última prueba construye la CLI en un directorio temporal, compila `full-language.cps`, ejecuta su ensamblador en MARS y compara stdout, stderr y éxito, sin modificar el fuente. Su salida esperada incluye Fibonacci, herencia, despacho virtual, alias, listas, foreach, capturas mutables, operación dinámica y recuperación de un índice inválido.

Para otro archivo:

```
go run ./cmd/compiscript-mips programa.cps > /tmp/programa.asm
bash scripts/mars.sh /tmp/programa.asm
```

Usá && o comprobá el código de salida antes de ejecutar un archivo redirigido: el shell puede crear un archivo vacío si la compilación falla.

CLI y reportes

```
go run ./cmd/compiscript-mips -h
go run ./cmd/compiscript-mips -format=json programa.cps
```

Interfaz	Comportamiento
`compiscript-mips [-format=mips|json] archivo.cps`	Ensamblador por defecto o reporte JSON con `mipsText`, TAC, diagnósticos y símbolos
`compiscript-tac`	Conserva su salida TAC/JSON anterior; no genera ni ejecuta MIPS
`compiscript`	Conserva su análisis AST anterior
`compiscript.CompileMIPS`	Fachada aditiva sobre `Compile`; no ejecuta Java ni evalúa el fuente

Las opciones preceden al archivo. Si necesitás observar directamente el código de salida de la CLI, construí un binario: `go build -o /tmp/compiscript-mips ./cmd/compiscript-mips`. El wrapper `go run` puede añadir `exit status N` y devolver su propio estado cuando falla el programa.

La CLI devuelve 0 al compilar, 1 por errores de fuente/lectura/generación y 2 por uso inválido. En formato MIPS no entrega ensamblador parcial: los diagnósticos van a stderr. En JSON entrega un reporte aun con errores y conserva salida 1; un error propio del backend usa fase `codegen`, código `MIPS_GENERATION` y ubicación cuando corresponde. El TAC exitosamente generado puede conservarse para inspección aunque falle el target; `mipsText` queda vacío.

IDE

```
go run ./cmd/ide
```

Abrí `http://localhost:8080`, cargá o escribí un `.cps` y elegí **Compile MIPS**. La pestaña **MIPS** muestra el ensamblador como texto; **Download .asm** descarga únicamente la última compilación válida de la fuente actual. Editar, cargar otra fuente, recibir un error o volver a compilar TAC invalida la descarga anterior. **Compile .cps (TAC)** y **Analyze .cps (AST)** conservan sus funciones previas.

El endpoint aditivo es `POST /api/compiscript/mips` con JSON `{"source":"print(1);"}`. Usa las mismas validaciones estrictas de transporte y límite de cuerpo que los endpoints anteriores. Solo compila: no hay un endpoint que ejecute ensamblador recibido. Para ejecutar lo descargado se usa MARS local.

El servidor existente escucha en `:8080`; usalo en una red local confiable. El IDE todavía depende de sus recursos CDN existentes para CodeMirror/Viz.js; el backend y MARS no necesitan CDN.

Arquitectura y alcance

```
.cps → ANTLR → AST → semántica/ámbitos → tac.Program
                                              ↓
                                       mips.Generate
                                              ↓
                                         .asm MIPS
                                              ↓
                                      MARS (separado)
```

El backend consume operandos, instrucciones, procedimientos y layouts estructurados; no vuelve a analizar texto TAC. Cubre escalares, aritmética y widening, cadenas UTF-8 con NUL, operadores y control, funciones y recursión, entornos/closures, objetos/herencia/métodos, listas, valores dinámicos y try/catch. Los errores de metadata interna se rechazan antes de devolver ensamblador; los fallos del lenguaje se verifican en ejecución y pueden ser capturados.

- ABI, valores, pila, registros y llamadas.
- Objetos, listas, contratos dinámicos y límites de impresión.
- Handlers, excepciones y desenrollado.

Los valores usan cajas etiquetadas y ubicaciones canónicas en memoria: el enfoque prioriza corrección y spilling sobre optimización. El frame preserva `$sp`/`$fp`, continuación y registros guardados; los argumentos se pasan por pila y el resultado por `$v0`. Los entornos escapados viven en heap.

Diagnóstico de problemas frecuentes

Síntoma	Qué revisar
`MARS no está instalado`	Ejecutar `bash scripts/install-mars.sh` desde el checkout correcto
Java ausente	Instalar/configurar Java por los medios del equipo; el instalador MARS no lo hace
Checksum inválido	No ejecutar el JAR; contrastar procedencia y `tools/mars/SHA256SUMS` antes de reemplazar la caché
Error `semantic` o `syntax`	Corregir el `.cps` en la ubicación indicada; no ejecutar un `.asm` vacío o anterior
Error `codegen MIPS_GENERATION`	Revisar mensaje/ubicación y límites del target; `mipsText` debe estar vacío
`format must be tac or json` al pedir MIPS	Se invocó la CLI anterior; usar `cmd/compiscript-mips`
El IDE no escucha en 8080	Revisar stdout/stderr y que el puerto no esté ocupado; no asumir que arrancó otro proceso
Editor web o diagramas no cargan	Verificar los CDN existentes y la conexión; compilar por CLI permite separar ese problema del backend
Download .asm está deshabilitado	Compilar MIPS sin errores para la fuente actual; editar/iniciar otra acción invalida el resultado previo
MARS produce un error de ensamblado/simulación	Separar ese mensaje de los diagnósticos del compilador; conservar `.cps`, `.asm`, stdout, stderr y salida para reproducir

Límites explícitos

- Target: MARS 4.5 en modo sin delayed branching (configuración por defecto del wrapper). No se afirma compatibilidad con toda ISA/configuración o ABI del sistema operativo.
- Números IEEE-754 binary64, como el oráculo TAC: no enteros arbitrariamente precisos. Literales no finitos o fuera de rango generan diagnóstico, no ensamblador truncado. Para valores no integrales o extremos, la representación decimal de MARS puede diferir de Go.
- Heap monotónico sin recolección de basura. Recursos finitos del simulador, stack/heap y terminación del programa no están garantizados.
- Frames, argumentos y layouts respetan desplazamientos inmediatos MIPS; un layout demasiado grande se rechaza explícitamente. Listas literales tienen el límite de layout documentado por el backend.
- Impresión de listas acotada a 64 valores/profundidad ocho y detección de ciclos. Objetos y funciones muestran IDs estables, no direcciones físicas.
- Un fallo no capturado imprime su mensaje en stdout y termina con código 1. Los mensajes propios de MARS van a stderr (incluye una línea vacía incluso en ejecución correcta); errores de ensamblado devuelven 1 y de simulación 2.

Pruebas y presentación

La evidencia de aceptación registra los 150 escenarios MIPS, la suite global, `vet`, las pruebas web, compatibilidad y límites de verificación.

```
go test ./internal/compiscript/mips -count=1 -v
go test ./internal/compiscript ./cmd/compiscript-mips ./cmd/compiscript-tac ./cmd/ide -count=1
node --test web/*.test.cjs
```

La suite MIPS ejecuta ensamblador real; `-short` omite esas ejecuciones externas. Las pruebas negativas verifican errores, ausencia de ensamblador parcial y preservación de datos después de rechazos donde corresponde. La suite global anterior puede regenerar ANTLR; seguí la política de reproducción y ejecutala en una copia aislada.

Recorrido sugerido para la presentación

1. Mostrar el informe y la matriz de requisitos sin prometer una nota.
2. Ejecutar `bash scripts/test-mips.sh` y abrir el fuente/salida esperada de la demo.
3. Mostrar TAC y MIPS de un ejemplo mínimo; explicar que MARS ejecuta el archivo, no el compilador.
4. Compilar desde el IDE y descargar un `.asm`; ejecutarlo con MARS por separado.
5. Mostrar una entrada inválida y que no se entrega ensamblador parcial.
6. Explicar el frame, el valor de retorno y una captura que sobrevive al frame, con la arquitectura y la referencia.
7. Cerrar con evidencia de pruebas, límites del target y contribuciones reales del equipo.

Antes de presentar, revisá también carga/edición/descarga en un navegador real y el entorno del evaluador. Los commits locales de implementación no demuestran por sí solos las contribuciones individuales de los tres integrantes; no se inventa autoría ni se publica automáticamente el trabajo.